Lab Report 3: Motors and Interfacing

Embedded Systems Control using MicroPython

Students: Yossi Brim & Elad Asbag

April 19, 2026

1 Introduction

This laboratory focuses on the integration and control of mechanical actuators using an ESP32 microcontroller and MicroPython. The experiments explore two fundamental motor types used in embedded systems: Servo motors and Stepper motors. The objectives are to implement analog-to-digital (ADC) mapping for angular control (PWM) and to develop a complex motion control system for a Stepper motor using an A4988 driver, emphasizing non-blocking programming logic.

2 Task 1: Potentiometer-Controlled Servo Motor

2.1 Objective and Methodology

The goal of this task is to establish a direct relationship between an analog input (potentiometer) and a mechanical output (servo angle). The system reads a 12-bit ADC value (0-4095) from the potentiometer and maps it to the appropriate PWM duty cycle values for a 50Hz frequency. We expect to see a smooth, real-time response where the motor arm follows the potentiometer's rotation.

2.2 Execution Proof

2.3 MicroPython Source Code

```python
from machine import Pin, PWM, ADC
import time

# ADC Configuration (Potentiometer on GPIO 34)
pot = ADC(Pin(34))
pot.atten(ADC.ATTN_11DB)

# PWM Configuration (Servo on GPIO 15, 50Hz)
servo = PWM(Pin(15), freq=50)

def map_value(x, in_min, in_max, out_min, out_max):
    return int((x - in_min) * (out_max - out_min) / (in_max - in_min) +
    out_min)

while True:
```

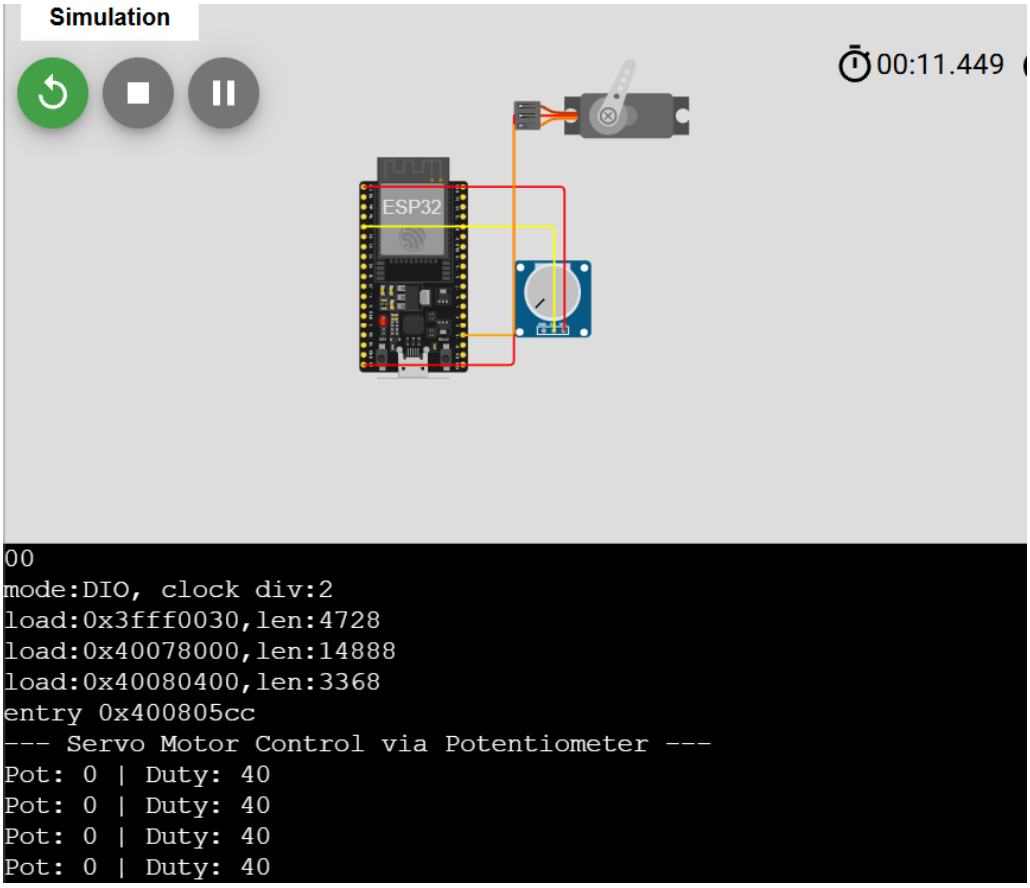

Figure 1: Task 1 Simulation: The terminal shows the mapping between ADC values and the PWM duty cycle, while the Servo arm reflects the input position.

```
    val = pot.read() # Reading ADC (0-4095)
    duty = map_value(val, 0, 4095, 40, 115) # Mapping to Servo Duty Cycle
    servo.duty(duty)
    print(f"Pot Value: {val} | Servo Duty: {duty}")
    time.sleep(0.05)
```

2.4 Analysis of Results

The system performed exactly as expected. Rotating the potentiometer resulted in an immediate change in the servo's angle. As seen in the simulation proof, when the potentiometer is at its minimum value (0), the motor stabilizes at a duty cycle of 40 (0°). The linear mapping ensures accuracy across the entire rotation range.

3 Task 2: Stepper Motor Control via A4988 Driver

3.1 Objective and Methodology

This task involves driving a stepper motor using an A4988 driver module combined with digital user inputs. The system is designed to execute a specific number of steps every 5 seconds while remaining responsive to two buttons: one for toggling the direction (CW/CCW) and another for selecting the step count (0, 10, 20, 30). The methodology utilizes `time.ticks_ms()` to implement non-blocking logic, ensuring the buttons are polled continuously during the wait intervals.

3.2 Execution Proof

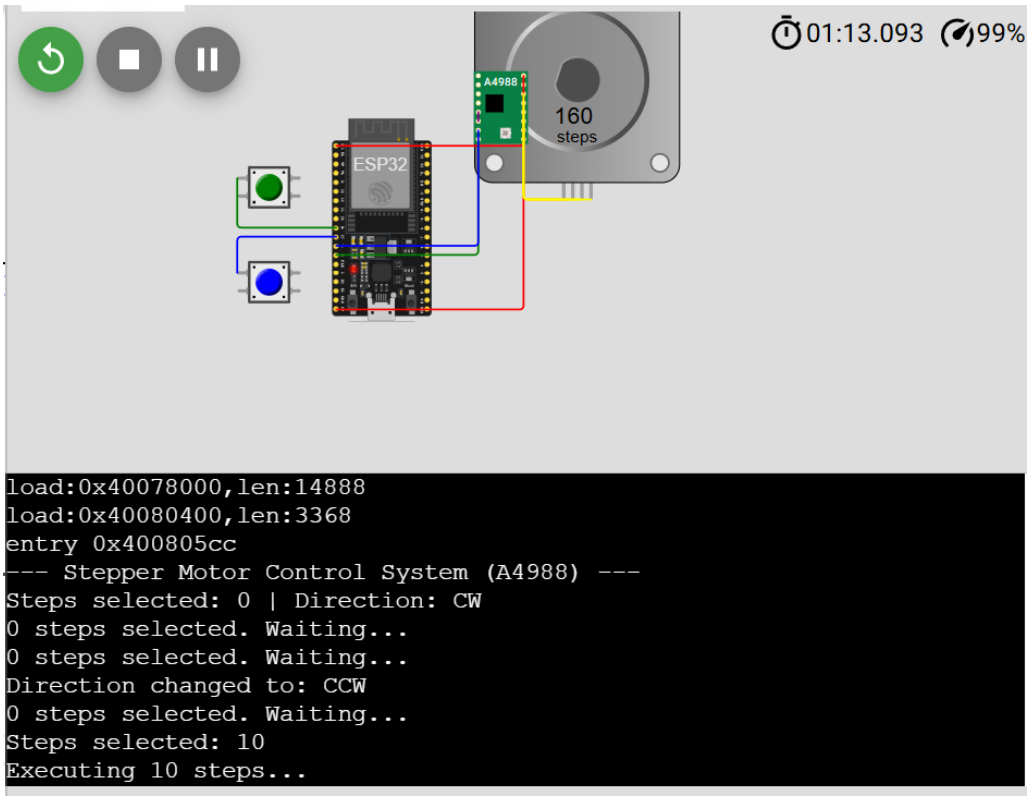

Figure 2: Task 2 Simulation: The terminal confirms real-time state changes and successful execution of 10 steps in CCW direction.

3.3 MicroPython Source Code

```
from machine import Pin
import time

# Pin Definitions
dir_pin = Pin(12, Pin.OUT)
step_pin = Pin(14, Pin.OUT)
btn_dir = Pin(26, Pin.IN, Pin.PULL_UP)
btn_steps = Pin(27, Pin.IN, Pin.PULL_UP)

```

```python
# State Variables
current_dir = 0
step_options = [0, 10, 20, 30]
idx = 0
last_move = time.ticks_ms()
last_btn_dir = 0
last_btn_steps = 0

def move_stepper(steps):
    for _ in range(steps):
        step_pin.value(1)
        time.sleep_us(1000)
        step_pin.value(0)
        time.sleep_us(1000)

while True:
    now = time.ticks_ms()

    # Non-blocking Direction Button Handling
    if btn_dir.value() == 0 and time.ticks_diff(now, last_btn_dir) > 250:
        current_dir = not current_dir
        dir_pin.value(current_dir)
        print(f"Direction changed: {'CW' if current_dir == 0 else 'CCW'}")
        last_btn_dir = now

    # Non-blocking Step Selection Button Handling
    if btn_steps.value() == 0 and time.ticks_diff(now, last_btn_steps) > 250:
        idx = (idx + 1) % 4
        print(f"Steps selected: {step_options[idx]}")
        last_btn_steps = now

    # Timed Execution (Every 5 seconds)
    if time.ticks_diff(now, last_move) >= 5000:
        s = step_options[idx]
        if s > 0:
            print(f"Executing {s} steps...")
            move_stepper(s)
        last_move = time.ticks_ms()
```

3.4 Analysis of Results

The implementation successfully met all requirements. The use of non-blocking timing logic was critical; the system remained perfectly responsive to button presses even during the 5-second idle periods. The simulation proof confirms that the motor correctly interpreted the CCW direction toggle and the 10-step command, executing them precisely on schedule.